

Programação Orientada a Objetos (POO) é um paradigma que organiza o software em objetos. Um objeto representa uma entidade do mundo real e possui atributos (dados) e métodos (comportamentos). Uma classe é um molde que define quais atributos e métodos os objetos terão. Quando criamos um objeto a partir de uma classe, dizemos que estamos instanciando essa classe utilizando a palavra-chave `new`. Os atributos armazenam informações sobre o objeto e os métodos definem as ações que ele pode executar. O construtor é um método especial utilizado para inicializar objetos; possui o mesmo nome da classe, não possui tipo de retorno e é executado automaticamente quando um objeto é criado.

Um dos pilares da POO é o encapsulamento. O encapsulamento consiste em proteger os atributos da classe, normalmente utilizando o modificador `private`, impedindo que sejam acessados diretamente por outras classes. Para acessar ou modificar esses atributos utilizam-se métodos `getters` e `setters`. `Getters` retornam valores dos atributos e `setters` permitem alterar seus valores. O encapsulamento aumenta a segurança do código e evita alterações indevidas nos dados internos dos objetos.

Outro conceito fundamental é a herança. A herança permite que uma classe reutilize atributos e métodos de outra classe. Em Java utiliza-se a palavra-chave `extends`. A classe que fornece os recursos é chamada de superclasse ou classe pai, enquanto a que herda é chamada de subclasse ou classe filha. A herança evita repetição de código e facilita manutenção. Java não permite herança múltipla entre classes, ou seja, uma classe só pode herdar diretamente de uma única classe. Entretanto, pode ocorrer herança em cascata: se C herda de B e B herda de A, então C também possui acesso aos recursos de A.

Polimorfismo é a capacidade de tratar objetos diferentes através de uma mesma referência. Um método pode existir em várias classes e apresentar comportamentos distintos dependendo do objeto que o executa. Relacionado a isso está a sobrescrita de métodos (`override`), realizada quando uma classe filha redefine um método herdado da classe pai. Para indicar isso utiliza-se a anotação `@Override`. Existe também a sobrecarga (`overload`), que ocorre quando existem vários métodos com o mesmo nome, porém com listas de parâmetros diferentes.

Classes abstratas são utilizadas quando se deseja criar uma classe base que não deve gerar objetos diretamente. Uma classe abstrata é declarada utilizando a palavra-chave `abstract` e não pode ser instanciada. Ela pode possuir atributos, construtores, métodos concretos e métodos abstratos. Métodos abstratos possuem apenas a assinatura, sem implementação, obrigando as subclasses a implementá-los. O objetivo é criar um modelo comum para diversas classes relacionadas. Por exemplo, uma classe abstrata `Animal` pode possuir atributos como nome e idade e um método abstrato `emitirSom()`. Classes como `Cachorro` e `Gato` herdam de `Animal` e implementam `emitirSom()` de formas diferentes. Classes abstratas são úteis quando existe compartilhamento de atributos e comportamentos entre várias classes.

Interfaces possuem finalidade semelhante às classes abstratas, mas funcionam como contratos. Uma interface define quais métodos uma classe deve implementar. Em Java utiliza-se a palavra-chave `interface` para criar uma interface e `implements` para implementá-la. Os métodos declarados em interfaces são, por padrão, públicos e abstratos. Diferentemente das classes abstratas, interfaces não possuem construtores. Uma classe pode implementar várias interfaces ao mesmo tempo, o que representa uma alternativa à ausência de herança múltipla em Java. Interfaces normalmente representam capacidades ou comportamentos. Por exemplo, uma interface `Imprimivel` pode obrigar qualquer classe que a implemente a possuir um método `imprimir()`. Conforme visto em aula, interfaces podem possuir atributos apenas na forma de constantes, ou seja, valores `public static final`. A principal diferença entre classe abstrata e interface é que a classe abstrata representa uma relação de herança e compartilhamento de implementação, enquanto a interface representa apenas um contrato de comportamento.

Exceções são mecanismos utilizados para tratar erros que ocorrem durante a execução do programa. Sem tratamento adequado, uma exceção pode encerrar a aplicação. Para lidar com esses erros utiliza-se a estrutura try-catch. O bloco try contém o código que pode gerar erro e o bloco catch captura e trata a exceção. É possível também utilizar a palavra-chave throw para lançar uma exceção manualmente e throws para informar que determinado método pode gerar uma exceção. Java possui diversas exceções prontas, mas também permite a criação de exceções personalizadas. Para criar uma exceção personalizada basta criar uma classe que herde de Exception. Esse recurso é muito utilizado quando se deseja representar erros específicos do sistema, como saldo insuficiente, saque inválido ou valor negativo. O método getMessage() permite recuperar a mensagem associada à exceção.

Na manipulação de arquivos, Java utiliza principalmente as classes do pacote java.io. Para escrita de arquivos são utilizadas as classes FileWriter e BufferedWriter. O FileWriter grava caracteres diretamente em um arquivo. Já o BufferedWriter adiciona um buffer de memória, melhorando significativamente o desempenho ao reduzir acessos ao disco. Geralmente os dois são utilizados juntos. Para leitura de arquivos utilizam-se FileReader e BufferedReader. O FileReader realiza a leitura básica de caracteres, enquanto o BufferedReader adiciona um buffer que torna a leitura mais eficiente. Durante operações de leitura e escrita podem ocorrer erros relacionados a permissões, arquivos inexistentes ou problemas de acesso ao disco. Esses erros normalmente geram uma IOException, que deve ser tratada através de try-catch. Ao finalizar a utilização de um arquivo é importante fechá-lo utilizando o método close().

Serialização é o processo de converter um objeto em uma sequência de bytes para que possa ser armazenado em arquivo, banco de dados, memória ou transmitido através da rede. O objetivo é preservar o estado de um objeto para recuperá-lo posteriormente. O processo inverso é chamado desserialização, no qual os bytes armazenados são transformados novamente em um objeto. Para que um objeto possa ser serializado sua classe deve implementar a interface Serializable. Durante a serialização são utilizados FileOutputStream e ObjectOutputStream. O FileOutputStream grava bytes em um arquivo e o ObjectOutputStream grava efetivamente o objeto. Na desserialização utilizam-se FileInputStream e ObjectInputStream. O FileInputStream lê os bytes armazenados e o ObjectInputStream reconstrói o objeto original. Um conceito importante é o modificador transient. Quando um atributo é declarado como transient ele não participa da serialização. Isso significa que seu valor não será salvo e, após a desserialização, retornará como null para objetos ou assumirá o valor padrão do tipo primitivo correspondente.

Além da serialização binária tradicional, existe a serialização em formatos textuais como XML e JSON. XML foi amplamente utilizado durante muitos anos e organiza informações através de tags. Embora seja bastante estruturado, tende a gerar arquivos maiores. JSON tornou-se o padrão mais utilizado atualmente devido à sua simplicidade, legibilidade e menor tamanho. JSON representa informações através de pares chave-valor e é amplamente utilizado em APIs, aplicações web e integração entre sistemas. Uma grande vantagem do JSON é sua independência de linguagem, permitindo que sistemas desenvolvidos em tecnologias diferentes troquem informações facilmente.

Sockets são utilizados para comunicação entre aplicações através de redes. Um socket representa um ponto final de comunicação entre dois programas. A comunicação ocorre normalmente entre um cliente e um servidor. O servidor permanece aguardando conexões enquanto o cliente inicia a comunicação. Quando a conexão é estabelecida, ambos podem enviar e receber informações, caracterizando uma comunicação bidirecional. A conexão é identificada por um endereço IP e uma porta. O endereço IP identifica o computador na rede e a porta identifica o serviço executado naquele computador.

No lado do servidor utiliza-se a classe `ServerSocket`. O servidor cria um `ServerSocket` associado a uma porta específica e utiliza o método `accept()` para aguardar conexões. O método `accept()` bloqueia a execução até que algum cliente tente se conectar. Quando a conexão é aceita, o servidor recebe um objeto `Socket` representando aquele cliente específico. A partir desse momento torna-se possível trocar dados entre cliente e servidor. Após concluir a comunicação, o servidor pode fechar a conexão utilizando `close()` e voltar a aguardar novos clientes.

No lado do cliente utiliza-se a classe `Socket`. O cliente cria um objeto `Socket` informando o endereço IP do servidor e a porta desejada. Se o servidor estiver disponível, a conexão é estabelecida e os dados podem ser trocados. O cliente normalmente envia uma solicitação, aguarda uma resposta e encerra a conexão. Em testes locais costuma-se utilizar o endereço `localhost`, que corresponde ao IP `127.0.0.1`, representando o próprio computador.

A comunicação através de sockets é realizada utilizando fluxos de entrada e saída. O método `getInputStream()` fornece um fluxo para receber dados e o método `getOutputStream()` fornece um fluxo para enviar dados. Quando a comunicação envolve tipos primitivos, como inteiros, doubles ou strings, normalmente utilizam-se `DataInputStream` e `DataOutputStream`. Esses objetos permitem transmitir dados já convertidos para formatos apropriados. Quando se deseja transmitir objetos completos, utilizam-se `ObjectInputStream` e `ObjectOutputStream`. Nesse caso, os objetos enviados devem implementar `Serializable`, pois precisam ser convertidos em bytes para atravessar a rede.

Um fluxo típico de comunicação cliente-servidor funciona da seguinte forma: o servidor cria um `ServerSocket` e aguarda conexões; o cliente cria um `Socket` e conecta-se ao servidor; o cliente envia dados; o servidor recebe esses dados, processa as informações e gera uma resposta; o servidor envia a resposta ao cliente; o cliente recebe a resposta e apresenta o resultado ao usuário; finalmente a conexão é encerrada. Esse modelo foi exatamente o utilizado nos exemplos da disciplina, tanto para envio de números inteiros quanto para envio de objetos serializados.

QUESTÃO 1 – Programação Orientada a Objetos

Desenvolva um sistema bancário utilizando **Programação Orientada a Objetos em Java**. O sistema deverá conter uma **classe abstrata** chamada `Conta`, contendo os atributos privados `titular` e `saldo`, além de um construtor para inicializar esses atributos, métodos `getters` e um método concreto para exibir os dados da conta. A classe `Conta` também deverá possuir dois **métodos abstratos**, `depositar(double valor)` e `sacar(double valor)`, que serão implementados pelas subclasses. Crie uma **interface** chamada `Tributavel`, contendo o método `calcularTaxa()`. Implemente duas classes concretas: `ContaCorrente`, que herda de `Conta` e implementa a interface `Tributavel`; `ContaPoupanca`, que herda de `Conta`. Crie uma **exceção personalizada** chamada `ValorInvalidoException`, utilizada para impedir depósitos e saques com valores menores ou iguais a zero e saques com valor superior ao saldo disponível. No método principal (`main`), instancie uma conta corrente e uma conta poupança utilizando **polimorfismo**, realize depósitos, saques, exiba os dados das contas e trate todas as exceções utilizando `try/catch`, apresentando a mensagem de erro através do método `getMessage()`.

```

public class Principal {
    public static void main(String[] args) {
        // Polimorfismo
        Conta c1 = new ContaCorrente("João", 1000);
        Conta c2 = new ContaPoupanca("Maria", 500);

        try {
            c1.depositar(300);
            c1.sacar(200);
            c1.exibirDados();

            System.out.println();

            c2.depositar(100);
            c2.sacar(50);
            c2.exibirDados();

            System.out.println();

            // Gera exceção
            c2.sacar(1000);
        } catch (ValorInvalidoException e) {
            System.out.println("Erro: " + e.getMessage());
        }
    }
}

// Interface
interface Tributavel {
    double calcularTaxa();
}

// Exceção personalizada
class ValorInvalidoException extends Exception {
    public ValorInvalidoException(String msg) {
        super(msg);
    }
}

// Classe abstrata
abstract class Conta {
    // Encapsulamento
    private String titular;
    private double saldo;

    // Construtor
    public Conta(String titular, double saldo) {
        this.titular = titular;
        this.saldo = saldo;
    }

    // Getters
    public String getTitular() {
        return titular;
    }

    public double getSaldo() {
        return saldo;
    }

    // Setter protegido
    protected void setSaldo(double saldo) {
        this.saldo = saldo;
    }

    // Método concreto
    public void exibirDados() {
        System.out.println("Titular: " + titular);
        System.out.println("Saldo: " + saldo);
    }

    // Métodos abstratos
    public abstract void depositar(double valor) throws ValorInvalidoException;
    public abstract void sacar(double valor) throws ValorInvalidoException;
}

// Classe filha
class ContaCorrente extends Conta implements Tributavel {
    public ContaCorrente(String titular, double saldo) {
        super(titular, saldo);
    }

    // Override
    @Override
    public void depositar(double valor) throws ValorInvalidoException {
        if (valor <= 0)
            throw new ValorInvalidoException("Valor inválido.");

        setSaldo(getSaldo() + valor);
    }

    // Override
    @Override
    public void sacar(double valor) throws ValorInvalidoException {
        if (valor <= 0)
            throw new ValorInvalidoException("Valor inválido.");

        if (valor > getSaldo())
            throw new ValorInvalidoException("Saldo insuficiente.");

        setSaldo(getSaldo() - valor);
    }

    // Método da interface
    @Override
    public double calcularTaxa() {
        return getSaldo() * 0.02;
    }

    // Override
    @Override
    public void exibirDados() {
        super.exibirDados();
        System.out.println("Taxa: " + calcularTaxa());
    }
}

// Outra classe filha
class ContaPoupanca extends Conta {
    public ContaPoupanca(String titular, double saldo) {
        super(titular, saldo);
    }

    @Override
    public void depositar(double valor) throws ValorInvalidoException {
        if (valor <= 0)
            throw new ValorInvalidoException("Valor inválido.");

        setSaldo(getSaldo() + valor);
    }

    @Override
    public void sacar(double valor) throws ValorInvalidoException {
        if (valor <= 0)
            throw new ValorInvalidoException("Valor inválido.");

        if (valor > getSaldo())
            throw new ValorInvalidoException("Saldo insuficiente.");

        setSaldo(getSaldo() - valor);
    }
}

```

2 Manipulação de Arquivos e Serialização

Desenvolva um sistema de cadastro de alunos em Java. O sistema deverá possuir uma classe Aluno contendo os atributos nome, idade e senha. O atributo senha não deverá ser serializado. No método principal, o programa deverá: cadastrar alguns alunos; os dados em um arquivo texto utilizando FileWriter e BufferedWriter; ler o arquivo utilizando FileReader e BufferedReader; serializar um objeto utilizando ObjectOutputStream; desserializar o objeto utilizando ObjectInputStream; demonstrar o uso do modificador transient; tratar todas as exceções utilizando try/catch.

```

import java.io.*;
// Classe serializável
class Aluno implements Serializable {
    private String nome;
    private int idade;
    // Não será serializado
    private transient String senha;
    // Construtor
    public Aluno(String nome, int idade, String senha) {
        this.nome = nome;
        this.idade = idade;
        this.senha = senha;
    }
    // Getters
    public String getNome() {
        return nome;
    }
    public int getIdade() {
        return idade;
    }
    public String getSenha() {
        return senha;
    }
    @Override
    public String toString() {
        return "Nome: " + nome +
            "\nIdade: " + idade +
            "\nSenha: " + senha;
    }
}

public class Principal {
    public static void main(String[] args) {
        // Cria objetos
        Aluno a1 = new Aluno("João", 20, "123");
        Aluno a2 = new Aluno("Maria", 22, "abc");
        // ESCRITA EM ARQUIVO TEXTO
        try {
            FileWriter fw = new FileWriter("alunos.txt");
            BufferedWriter bw = new BufferedWriter(fw);
            bw.write(a1.getNome() + ", " + a1.getIdade());
            bw.newLine();
            bw.write(a2.getNome() + ", " + a2.getIdade());
            bw.close();
            fw.close();
        }
    }
}

```

```

        System.out.println("Arquivo criado!");
    } catch (IOException e) {
        System.out.println(e.getMessage());
    }
    // LEITURA DO ARQUIVO
    try {
        FileReader fr = new FileReader("alunos.txt");
        BufferedReader br = new BufferedReader(fr);
        String linha;
        while ((linha = br.readLine()) != null) {
            System.out.println(linha);
        }
        br.close();
        fr.close();
    } catch (IOException e) {
        System.out.println(e.getMessage());
    }
    // SERIALIZAÇÃO
    try {
        FileOutputStream fos =
            new FileOutputStream("aluno.ser");
        ObjectOutputStream oos =
            new ObjectOutputStream(fos);
        oos.writeObject(a1);
        oos.close();
        fos.close();
        System.out.println("\nObjeto serializado!");
    } catch (IOException e) {
        System.out.println(e.getMessage());
    }
    // DESSERIALIZAÇÃO
    try {
        FileInputStream fis =
            new FileInputStream("aluno.ser");
        ObjectInputStream ois =
            new ObjectInputStream(fis);
        Aluno aluno =
            (Aluno) ois.readObject();
        ois.close();
        fis.close();
        System.out.println("\nObjeto recuperado:");
        System.out.println(aluno);
        // Como é transient
        System.out.println("Senha: "
            + aluno.getSenha());
    }
}

```

```

    try {
        FileInputStream fis =
            new FileInputStream("aluno.ser");
        ObjectInputStream ois =
            new ObjectInputStream(fis);
        Aluno aluno =
            (Aluno) ois.readObject();
        ois.close();
        fis.close();
        System.out.println("\nObjeto recuperado:");
        System.out.println(aluno);
        // Como é transient
        System.out.println("Senha: "
            + aluno.getSenha());
    }
    catch (IOException e) {
        System.out.println(e.getMessage());
    }
    catch (ClassNotFoundException e) {
        System.out.println(e.getMessage());
    }
}
}

```